

SECTION 2

ADDRESSING CAPABILITIES

Most operations take a source operand and destination operand, compute them, and store the result in the destination location. Single-operand operations take a destination operand, compute it, and store the result in the destination location. External microprocessor references to memory are either program references that refer to program space or data references that refer to data space. They access either instruction words or operands (data items) for an instruction. Program space is the section of memory that contains the program instructions and any immediate data operands residing in the instruction stream. Data space is the section of memory that contains the program data. Data items in the instruction stream can be accessed with the program counter relative addressing modes; these accesses classify as program references.

2.1 INSTRUCTION FORMAT

M68000 family instructions consist of at least one word; some have as many as 11 words. Figure 2-1 illustrates the general composition of an instruction. The first word of the instruction, called the simple effective address operation word, specifies the length of the instruction, the effective addressing mode, and the operation to be performed. The remaining words, called brief and full extension words, further specify the instruction and operands. These words can be floating-point command words, conditional predicates, immediate operands, extensions to the effective addressing mode specified in the simple effective address operation word, branch displacements, bit number or bit field specifications, special register specifications, trap operands, pack/unpack constants, or argument counts.

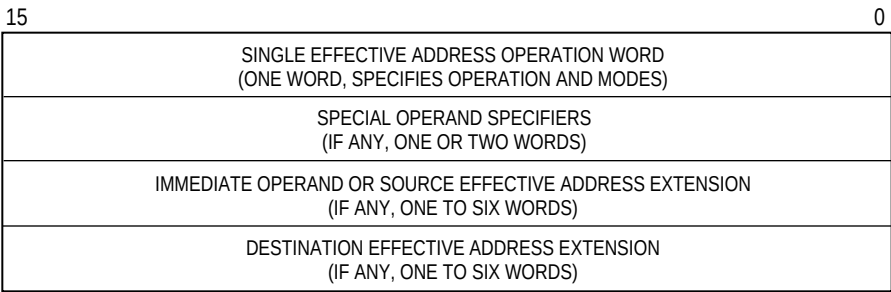


Figure 2-1. Instruction Word General Format

An instruction specifies the function to be performed with an operation code and defines the location of every operand. Instructions specify an operand location by register specification, the instruction's register field holds the register's number; by effective address, the instruction's effective address field contains addressing mode information; or by implicit reference, the definition of the instruction implies the use of specific registers.

The single effective address operation word format is the basic instruction word (see Figure 2-2). The encoding of the mode field selects the addressing mode. The register field contains the general register number or a value that selects the addressing mode when the mode field contains opcode 111. Some indexed or indirect addressing modes use a combination of the simple effective address operation word followed by a brief extension word. Other indexed or indirect addressing modes consist of the simple effective address operation word and a full extension word. The longest instruction is a MOVE instruction with a full extension word for both the source and destination effective addresses and eight other extension words. It also contains 32-bit base displacements and 32-bit outer displacements for both source and destination addresses. Figure 2-2 illustrates the three formats used in an instruction word; Table 2-1 lists the field definitions for these three formats.

SINGLE EFFECTIVE ADDRESS OPERATION WORD FORMAT

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
X	X	X	X	X	X	X	X	X	X	EFFECTIVE ADDRESS					
										MODE			REGISTER		

BRIEF EXTENSION WORD FORMAT

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D/A	REGISTER			W/L	SCALE		0	DISPLACEMENT							

FULL EXTENSION WORD FORMAT

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D/A	REGISTER			W/L	SCALE		1	BS	IS	BD SIZE		0	I/IS		
BASE DISPLACEMENT (0, 1, OR 2 WORDS)															
OUTER DISPLACEMENT (0, 1, OR 2 WORDS)															

Figure 2-2. Instruction Word Specification Formats

Table 2-1. Instruction Word Format Field Definitions

Field	Definition
Instruction	
Mode	Addressing Mode
Register	General Register Number
Extensions	
D/A	Index Register Type 0 = Dn 1 = An
W/L	Word/Long-Word Index Size 0 = Sign-Extended Word 1 = Long Word
Scale	Scale Factor 00 = 1 01 = 2 10 = 4 11 = 8
BS	Base Register Suppress 0 = Base Register Added 1 = Base Register Suppressed
IS	Index Suppress 0 = Evaluate and Add Index Operand 1 = Suppress Index Operand
BD SIZE	Base Displacement Size 00 = Reserved 01 = Null Displacement 10 = Word Displacement 11 = Long Displacement
I/IS	Index/Indirect Selection Indirect and Indexing Operand Determined in Conjunction with Bit 6, Index Suppress

For effective addresses that use a full extension word format, the index suppress (IS) bit and the index/indirect selection (I/IS) field determine the type of indexing and indirect action. Table 2-2 lists the index and indirect operations corresponding to all combinations of IS and I/IS values.

Table 2-2. IS-I/IS Memory Indirect Action Encodings

IS	Index/Indirect	Operation
0	000	No Memory Indirect Action
0	001	Indirect Preindexed with Null Outer Displacement
0	010	Indirect Preindexed with Word Outer Displacement
0	011	Indirect Preindexed with Long Outer Displacement
0	100	Reserved
0	101	Indirect Postindexed with Null Outer Displacement
0	110	Indirect Postindexed with Word Outer Displacement
0	111	Indirect Postindexed with Long Outer Displacement
1	000	No Memory Indirect Action
1	001	Memory Indirect with Null Outer Displacement
1	010	Memory Indirect with Word Outer Displacement
1	011	Memory Indirect with Long Outer Displacement
1	100–111	Reserved

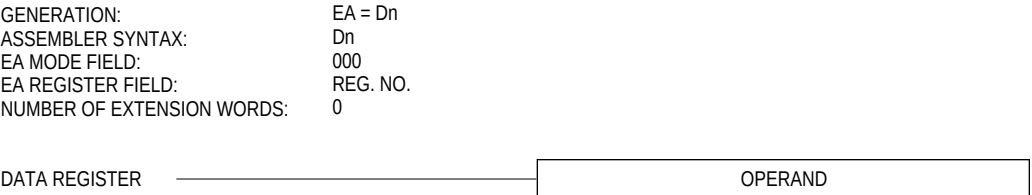
2.2 EFFECTIVE ADDRESSING MODES

Besides the operation code, which specifies the function to be performed, an instruction defines the location of every operand for the function. Instructions specify an operand location in one of three ways. A register field within an instruction can specify the register to be used; an instruction's effective address field can contain addressing mode information; or the instruction's definition can imply the use of a specific register. Other fields within the instruction specify whether the register selected is an address or data register and how the register is to be used. **Section 1 Introduction** contains detailed register descriptions.

An instruction's addressing mode specifies the value of an operand, a register that contains the operand, or how to derive the effective address of an operand in memory. Each addressing mode has an assembler syntax. Some instructions imply the addressing mode for an operand. These instructions include the appropriate fields for operands that use only one addressing mode.

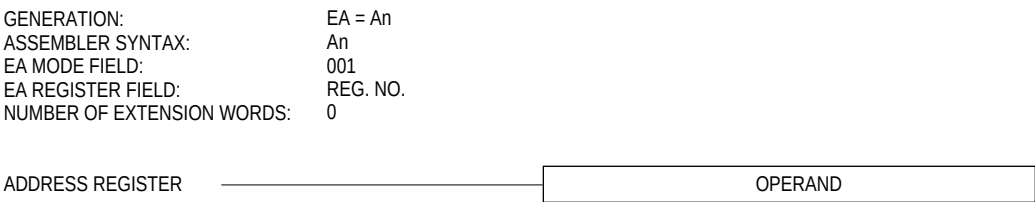
2.2.1 Data Register Direct Mode

In the data register direct mode, the effective address field specifies the data register containing the operand.



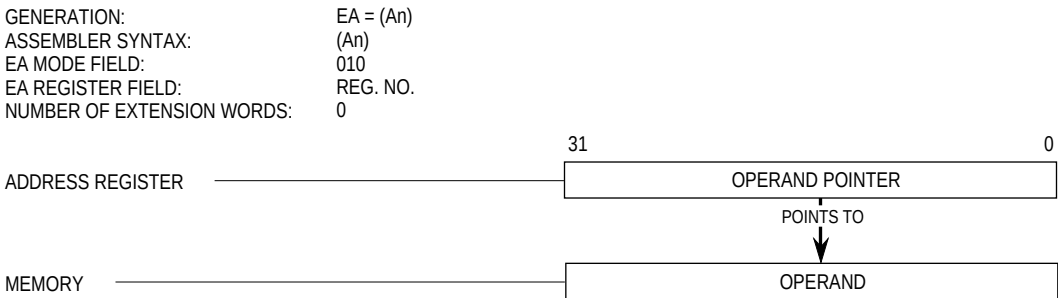
2.2.2 Address Register Direct Mode

In the address register direct mode, the effective address field specifies the address register containing the operand.



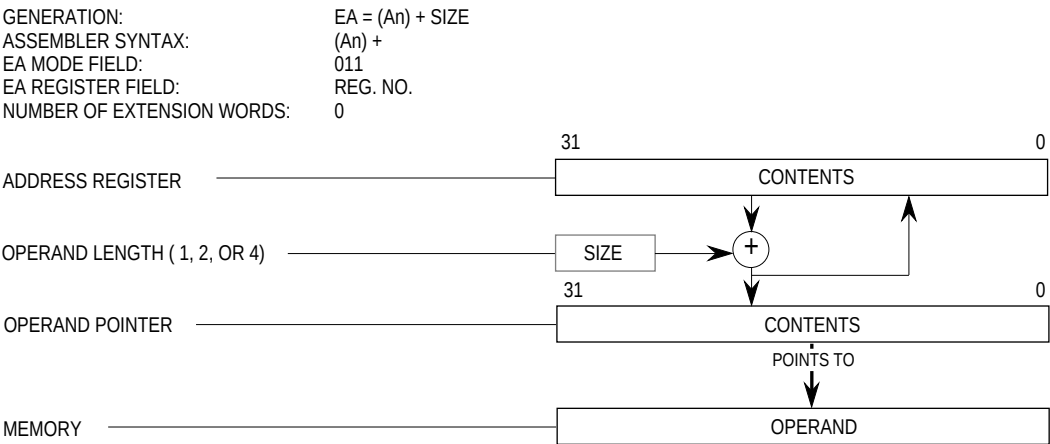
2.2.3 Address Register Indirect Mode

In the address register indirect mode, the operand is in memory. The effective address field specifies the address register containing the address of the operand in memory.



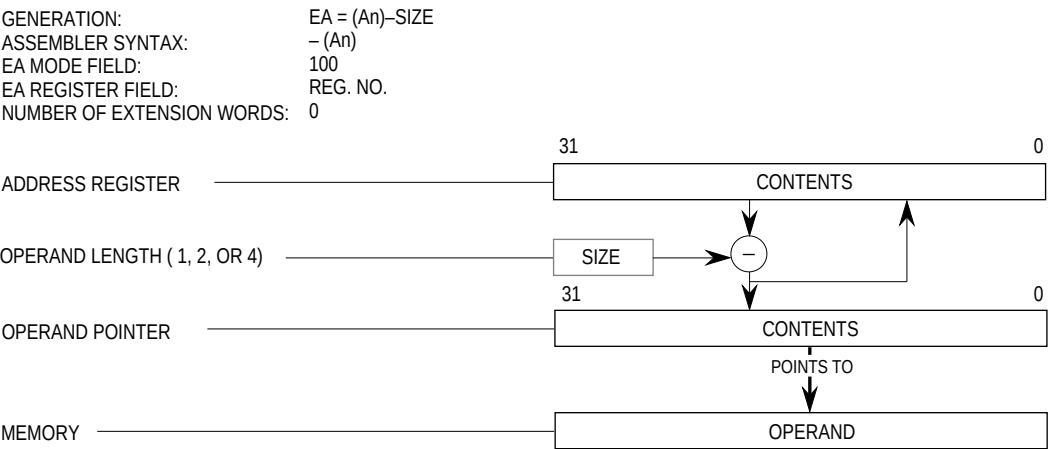
2.2.4 Address Register Indirect with Postincrement Mode

In the address register indirect with postincrement mode, the operand is in memory. The effective address field specifies the address register containing the address of the operand in memory. After the operand address is used, it is incremented by one, two, or four depending on the size of the operand: byte, word, or long word, respectively. Coprocessors may support incrementing for any operand size, up to 255 bytes. If the address register is the stack pointer and the operand size is byte, the address is incremented by two to keep the stack pointer aligned to a word boundary.



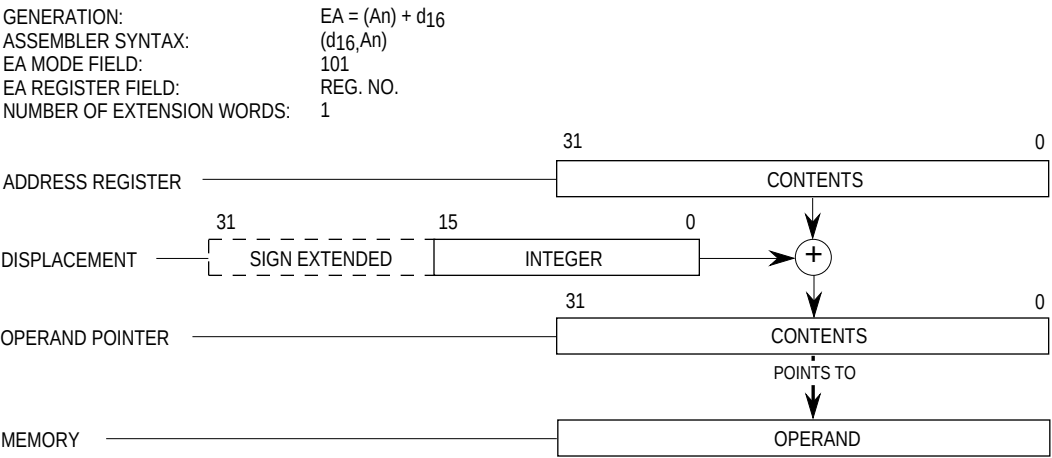
2.2.5 Address Register Indirect with Predecrement Mode

In the address register indirect with predecrement mode, the operand is in memory. The effective address field specifies the address register containing the address of the operand in memory. Before the operand address is used, it is decremented by one, two, or four depending on the operand size: byte, word, or long word, respectively. Coprocessors may support decrementing for any operand size up to 255 bytes. If the address register is the stack pointer and the operand size is byte, the address is decremented by two to keep the stack pointer aligned to a word boundary.



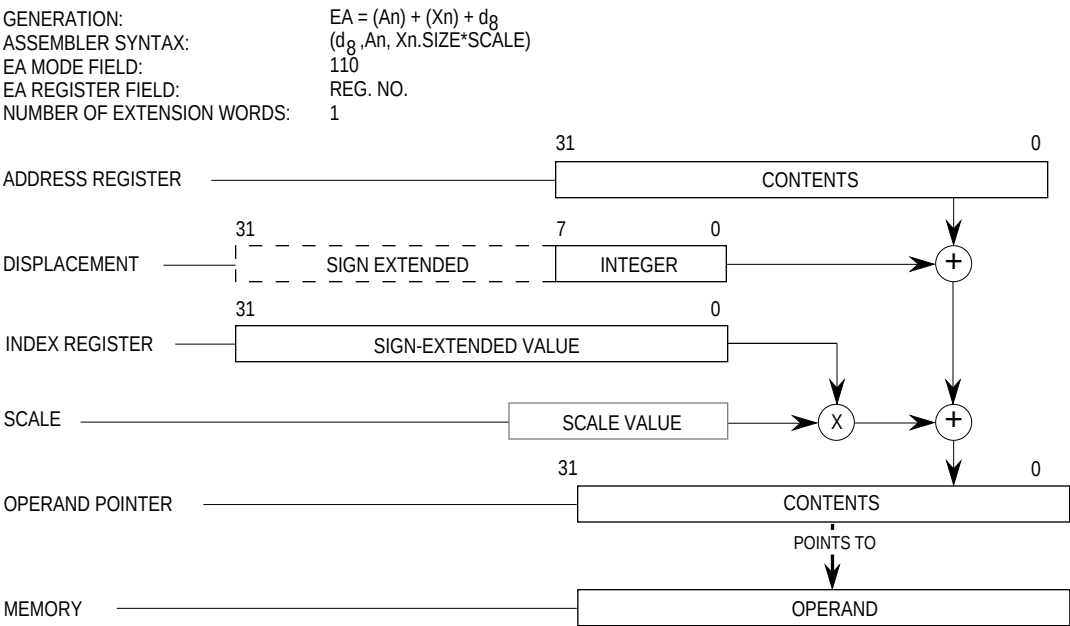
2.2.6 Address Register Indirect with Displacement Mode

In the address register indirect with displacement mode, the operand is in memory. The sum of the address in the address register, which the effective address specifies, plus the sign-extended 16-bit displacement integer in the extension word is the operand's address in memory. Displacements are always sign-extended to 32 bits prior to being used in effective address calculations.



2.2.7 Address Register Indirect with Index (8-Bit Displacement) Mode

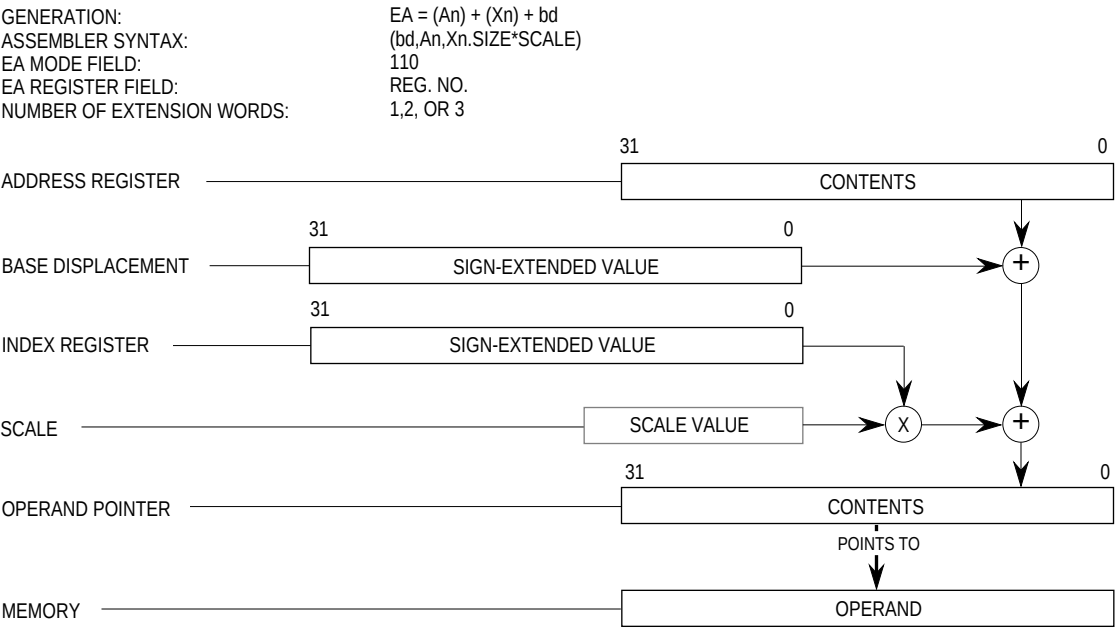
This addressing mode requires one extension word that contains an index register indicator and an 8-bit displacement. The index register indicator includes size and scale information. In this mode, the operand is in memory. The operand's address is the sum of the address register's contents; the sign-extended displacement value in the extension word's low-order eight bits; and the index register's sign-extended contents (possibly scaled). The user must specify the address register, the displacement, and the index register in this mode.



2.2.8 Address Register Indirect with Index (Base Displacement) Mode

This addressing mode requires an index register indicator and an optional 16- or 32-bit sign-extended base displacement. The index register indicator includes size and scaling information. The operand is in memory. The operand's address is the sum of the contents of the address register, the base displacement, and the scaled contents of the sign-extended index register.

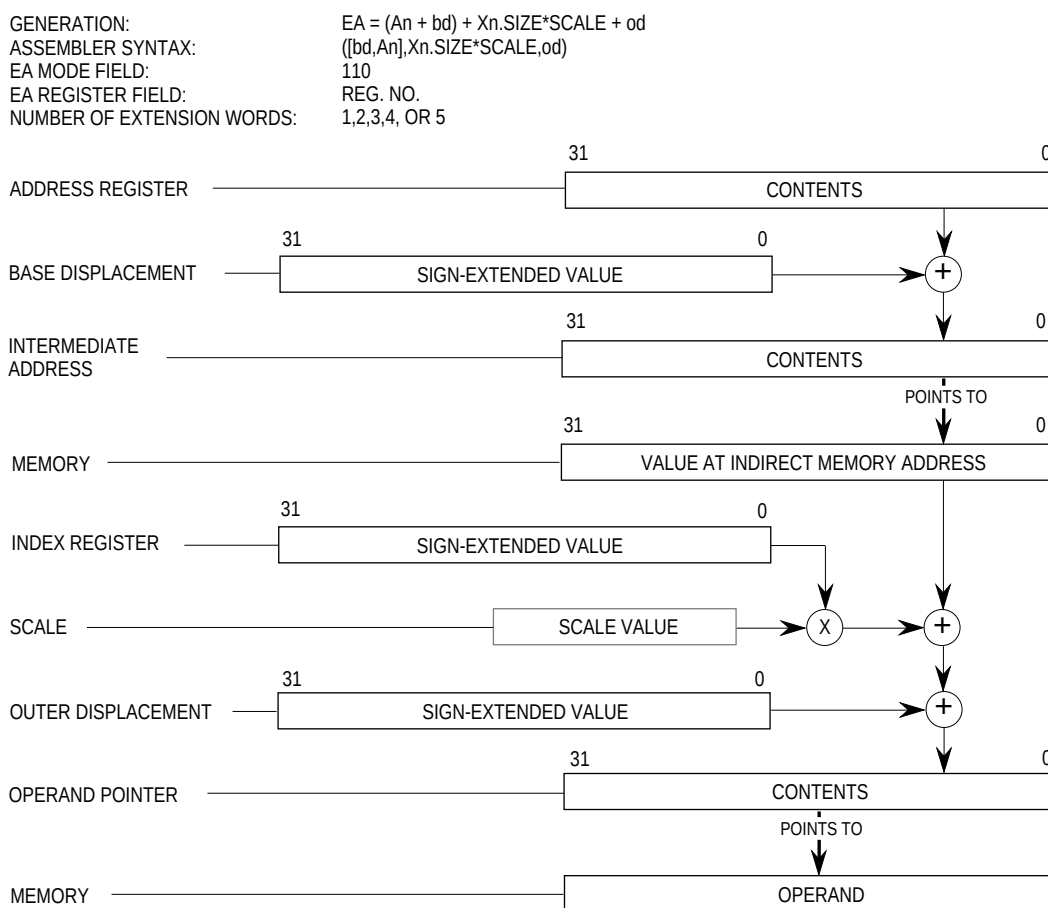
In this mode, the address register, the index register, and the displacement are all optional. The effective address is zero if there is no specification. This mode provides a data register indirect address when there is no specific address register and the index register is a data register.



2.2.9 Memory Indirect Postindexed Mode

In this mode, both the operand and its address are in memory. The processor calculates an intermediate indirect memory address using a base address register and base displacement. The processor accesses a long word at this address and adds the index operand ($Xn.SIZE * SCALE$) and the outer displacement to yield the effective address. Both displacements and the index register contents are sign-extended to 32 bits.

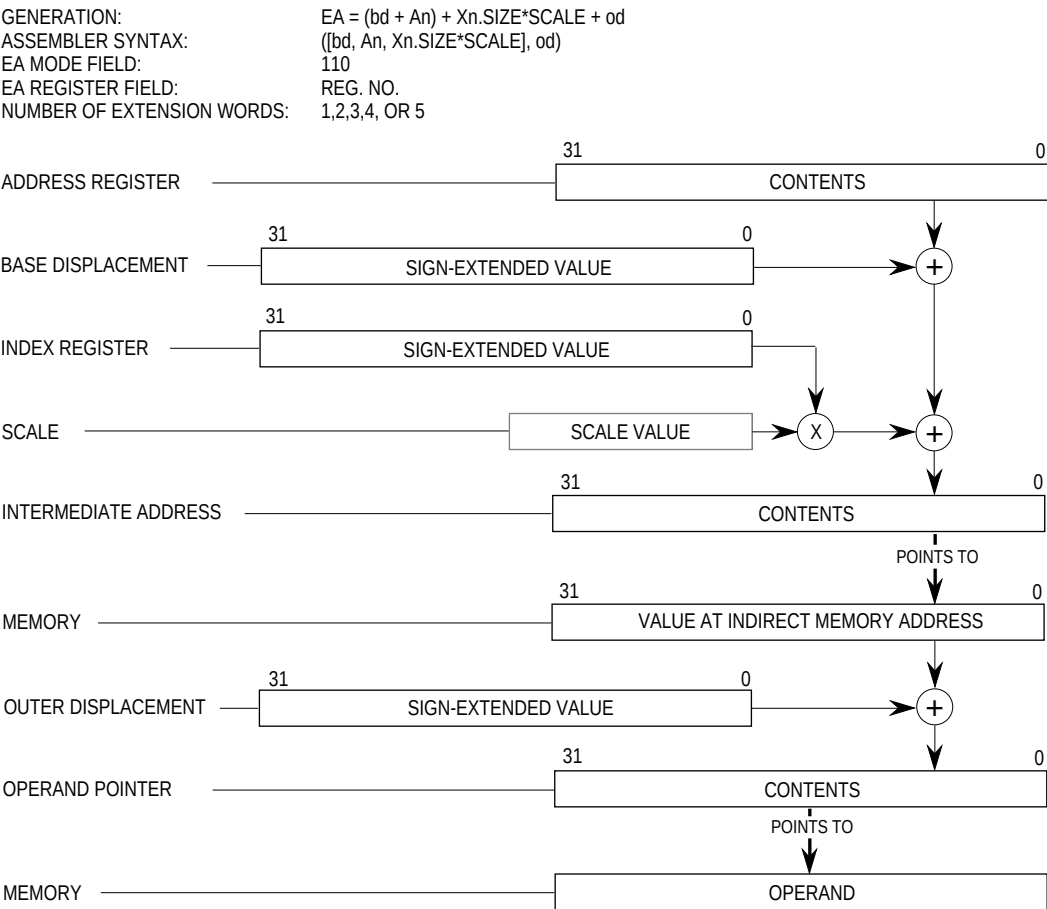
In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. Both the base and outer displacements may be null, word, or long word. When omitting a displacement or suppressing an element, its value is zero in the effective address calculation.



2.2.10 Memory Indirect Preindexed Mode

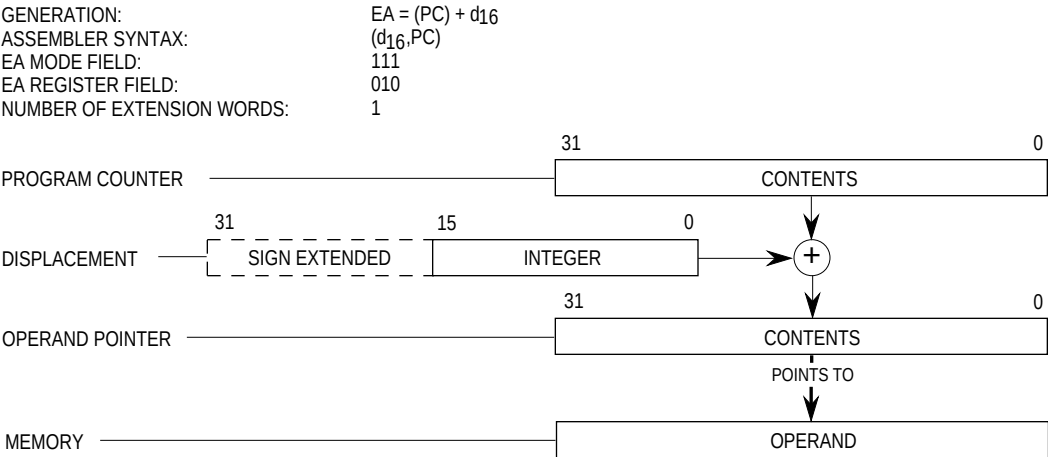
In this mode, both the operand and its address are in memory. The processor calculates an intermediate indirect memory address using a base address register, a base displacement, and the index operand ($Xn.SIZE*SCALE$). The processor accesses a long word at this address and adds the outer displacement to yield the effective address. Both displacements and the index register contents are sign-extended to 32 bits.

In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. Both the base and outer displacements may be null, word, or long word. When omitting a displacement or suppressing an element, its value is zero in the effective address calculation.



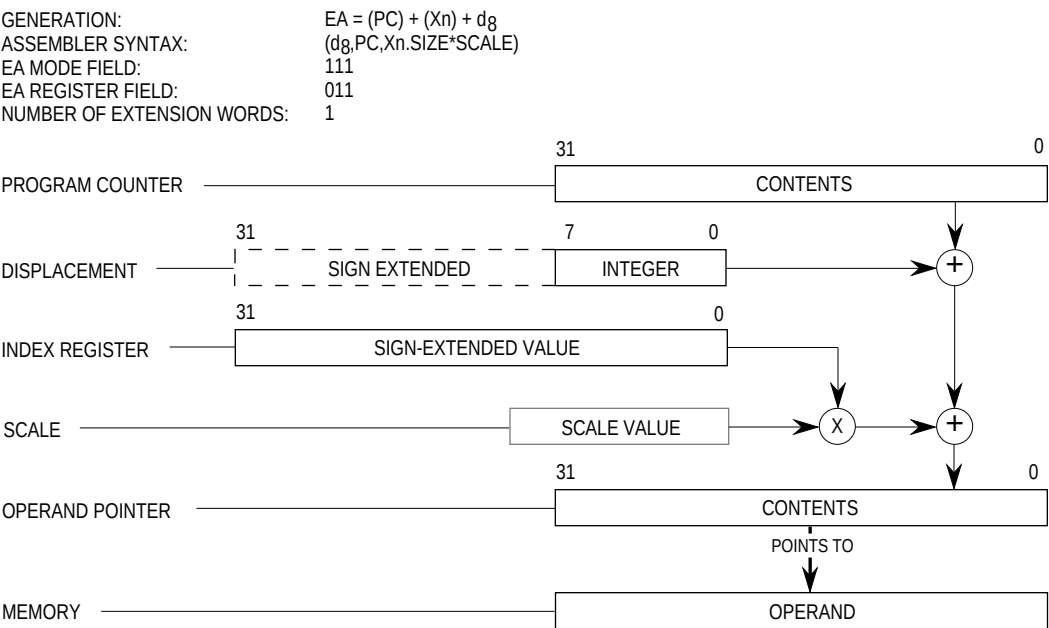
2.2.11 Program Counter Indirect with Displacement Mode

In this mode, the operand is in memory. The address of the operand is the sum of the address in the program counter (PC) and the sign-extended 16-bit displacement integer in the extension word. The value in the PC is the address of the extension word. This is a program reference allowed only for reads.



2.2.12 Program Counter Indirect with Index (8-Bit Displacement) Mode

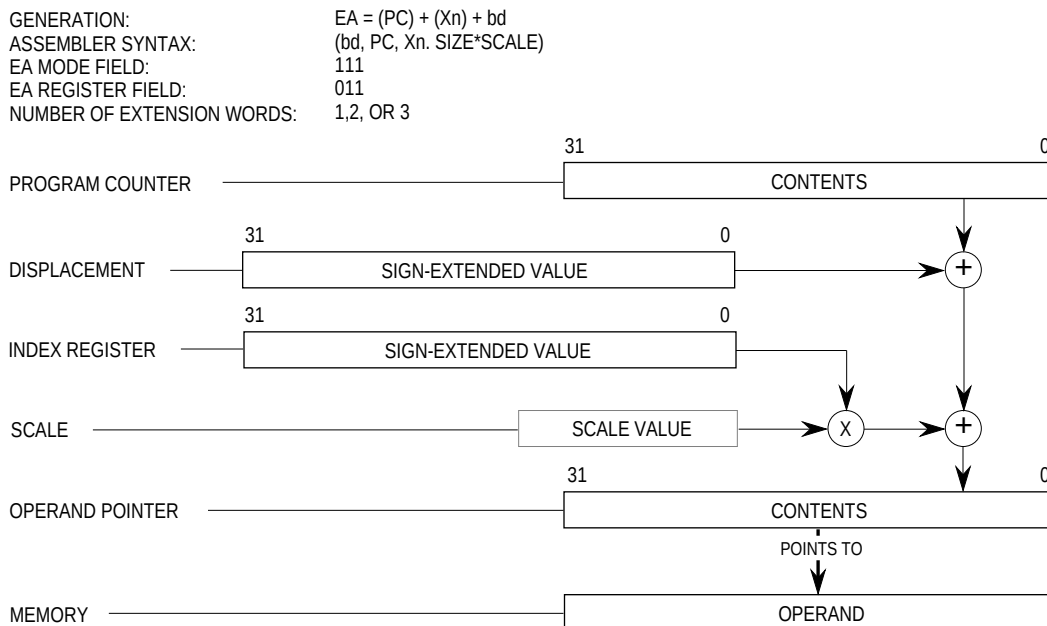
This mode is similar to the mode described in **2.2.7 Address Register Indirect with Index (8-Bit Displacement) Mode**, except the PC is the base register. The operand is in memory. The operand's address is the sum of the address in the PC, the sign-extended displacement integer in the extension word's lower eight bits, and the sized, scaled, and sign-extended index operand. The value in the PC is the address of the extension word. This is a program reference allowed only for reads. The user must include the displacement, the PC, and the index register when specifying this addressing mode.



2.2.13 Program Counter Indirect with Index (Base Displacement) Mode

This mode is similar to the mode described in **2.2.8 Address Register Indirect with Index (Base Displacement) Mode**, except the PC is the base register. It requires an index register indicator and an optional 16- or 32-bit sign-extended base displacement. The operand is in memory. The operand's address is the sum of the contents of the PC, the base displacement, and the scaled contents of the sign-extended index register. The value of the PC is the address of the first extension word. This is a program reference allowed only for reads.

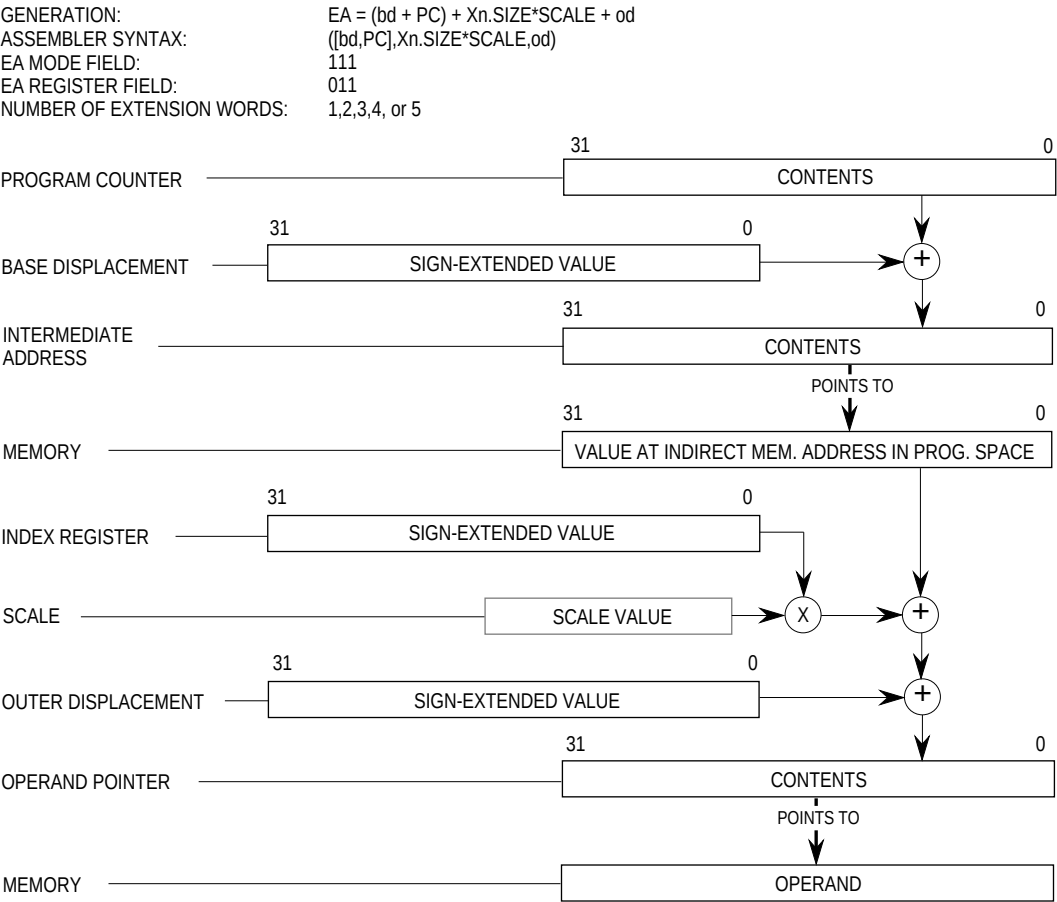
In this mode, the PC, the displacement, and the index register are optional. The user must supply the assembler notation ZPC (a zero value PC) to show that the PC is not used. This allows the user to access the program space without using the PC in calculating the effective address. The user can access the program space with a data register indirect access by placing ZPC in the instruction and specifying a data register as the index register.



2.2.14 Program Counter Memory Indirect Postindexed Mode

This mode is similar to the mode described in **2.2.9 Memory Indirect Postindexed Mode**, but the PC is the base register. Both the operand and operand address are in memory. The processor calculates an intermediate indirect memory address by adding a base displacement to the PC contents. The processor accesses a long word at that address and adds the scaled contents of the index register and the optional outer displacement to yield the effective address. The value of the PC used in the calculation is the address of the first extension word. This is a program reference allowed only for reads.

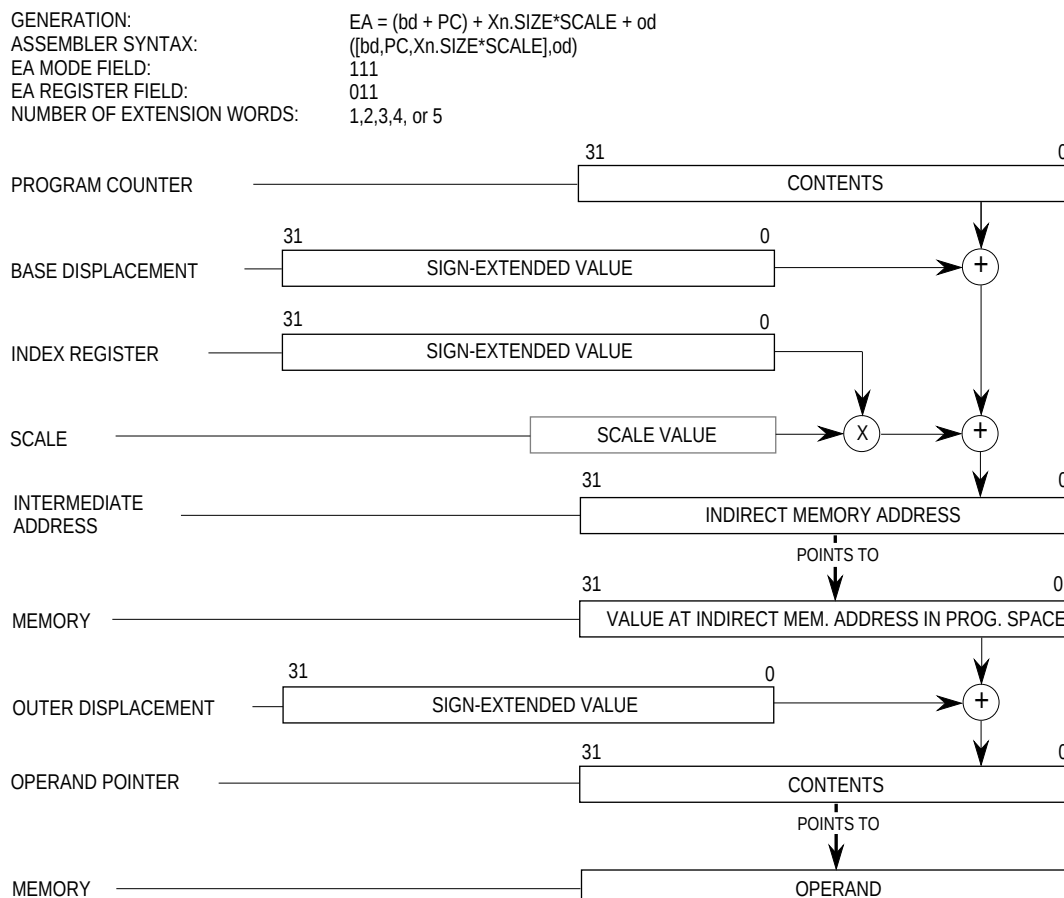
In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. The user must supply the assembler notation ZPC (a zero value PC) to show the PC is not used. This allows the user to access the program space without using the PC in calculating the effective address. Both the base and outer displacements may be null, word, or long word. When omitting a displacement or suppressing an element, its value is zero in the effective address calculation.



2.2.15 Program Counter Memory Indirect Preindexed Mode

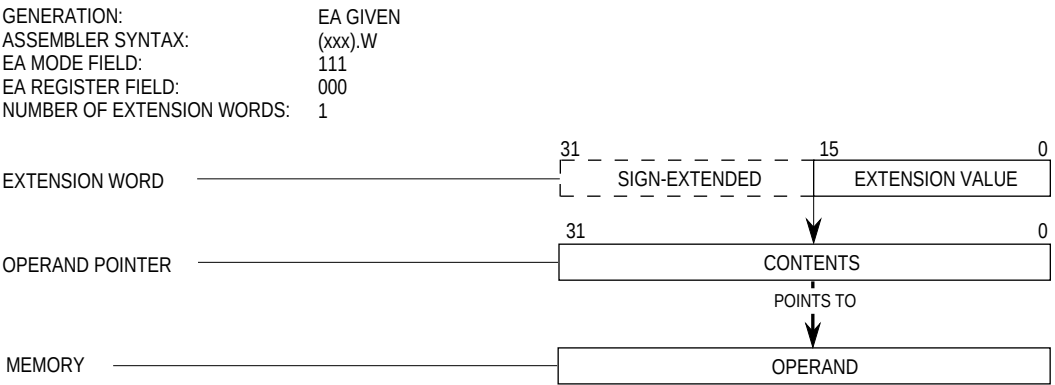
This mode is similar to the mode described in **2.2.10 Memory Indirect Preindexed Mode**, but the PC is the base register. Both the operand and operand address are in memory. The processor calculates an intermediate indirect memory address by adding the PC contents, a base displacement, and the scaled contents of an index register. The processor accesses a long word at immediate indirect memory address and adds the optional outer displacement to yield the effective address. The value of the PC is the address of the first extension word. This is a program reference allowed only for reads.

In the syntax for this mode, brackets enclose the values used to calculate the intermediate memory address. All four user-specified values are optional. The user must supply the assembler notation ZPC showing that the PC is not used. This allows the user to access the program space without using the PC in calculating the effective address. Both the base and outer displacements may be null, word, or long word. When omitting a displacement or suppressing an element, its value is zero in the effective address calculation.



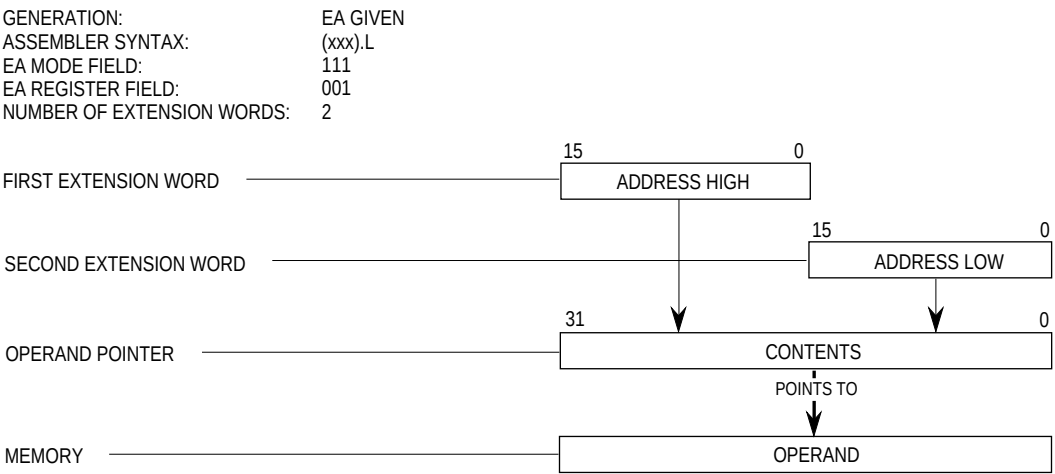
2.2.16 Absolute Short Addressing Mode

In this addressing mode, the operand is in memory, and the address of the operand is in the extension word. The 16-bit address is sign-extended to 32 bits before it is used. .



2.2.17 Absolute Long Addressing Mode

In this addressing mode, the operand is in memory, and the operand's address occupies the two extension words following the instruction word in memory. The first extension word contains the high-order part of the address; the second contains the low-order part of the address. .



2.2.18 Immediate Data

In this addressing mode, the operand is in one or two extension words. Table 2-3 lists the location of the operand within the instruction word format. The immediate data format is as follows:

GENERATION:	OPERAND GIVEN
ASSEMBLER SYNTAX:	#<xxx>
EA MODE FIELD:	111
EA REGISTER FIELD:	100
NUMBER OF EXTENSION WORDS:	1,2,4, OR 6, EXCEPT FOR PACKED DECIMAL REAL OPERANDS

Table 2-3. Immediate Operand Location

Operation Length	Location
Byte	Low-order byte of the extension word.
Word	The entire extension word.
Long Word	High-order word of the operand is in the first extension word; the low-order word is in the second extension word.
Single-Precision	In two extension words.
Double-Precision	In four extension words.
Extended-Precision	In six extension words.
Packed-Decimal Real	In six extension words.

2.3 EFFECTIVE ADDRESSING MODE SUMMARY

Effective addressing modes are grouped according to the use of the mode. Data addressing modes refer to data operands. Memory addressing modes refer to memory operands. Alterable addressing modes refer to alterable (writable) operands. Control addressing modes refer to memory operands without an associated size.

These categories sometimes combine to form new categories that are more restrictive. Two combined classifications are alterable memory (addressing modes that are both alterable and memory addresses) and data alterable (addressing modes that are both alterable and data). Table 2-4 lists a summary of effective addressing modes and their categories.

Table 2-4. Effective Addressing Modes and Categories

Addressing Modes	Syntax	Mode Field	Reg. Field	Data	Memory	Control	Alterable
Register Direct							
Data	Dn	000	reg. no.	X	—	—	X
Address	An	001	reg. no.	—	—	—	X
Register Indirect							
Address	(An)	010	reg. no.	X	X	X	X
Address with Postincrement	(An)+	011	reg. no.	X	X	—	X
Address with Predecrement	–(An)	100	reg. no.	X	X	—	X
Address with Displacement	(d ₁₆ ,An)	101	reg. no.	X	X	X	X
Address Register Indirect with Index							
8-Bit Displacement	(d ₈ ,An,Xn)	110	reg. no.	X	X	X	X
Base Displacement	(bd,An,Xn)	110	reg. no.	X	X	X	X
Memory Indirect							
Postindexed	([bd,An],Xn,od)	110	reg. no.	X	X	X	X
Preindexed	([bd,An,Xn],od)	110	reg. no.	X	X	X	X
Program Counter Indirect with Displacement	(d ₁₆ ,PC)	111	010	X	X	X	—
Program Counter Indirect with Index							
8-Bit Displacement	(d ₈ ,PC,Xn)	111	011	X	X	X	—
Base Displacement	(bd,PC,Xn)	111	011	X	X	X	—
Program Counter Memory Indirect							
Postindexed	([bd,PC],Xn,od)	111	011	X	X	X	X
Preindexed	([bd,PC,Xn],od)	111	011	X	X	X	X
Absolute Data Addressing							
Short	(xxx).W	111	000	X	X	X	—
Long	(xxx).L	111	000	X	X	X	—
Immediate	#<xxx>	111	100	X	X	—	—

2.4 BRIEF EXTENSION WORD FORMAT COMPATIBILITY

Programs can be easily transported from one member of the M68000 family to another in an upward-compatible fashion. The user object code of each early member of the family, which is upward compatible with newer members, can be executed on the newer microprocessor without change. Brief extension word formats are encoded with information that allows the CPU32, MC68020, MC68030, and MC68040 to distinguish the basic M68000 family architecture's new address extensions. Figure 2-3 illustrates these brief extension word formats. The encoding for SCALE used by the CPU32, MC68020, MC68030, and MC68040 is a compatible extension of the M68000 family architecture. A value of zero for SCALE is the same encoding for both extension words. Software that uses this encoding is compatible with all processors in the M68000 family. Both brief extension word formats do not contain the other values of SCALE. Software can be easily migrated in an upward-compatible direction, with downward support only for nonscaled addressing. If the MC68000 were to execute an instruction that encoded a scaling factor, the scaling factor would be ignored and would not access the desired memory address. The earlier microprocessors do not recognize the brief extension word formats implemented by newer processors. Although they can detect illegal instructions, they do not decode invalid encodings of the brief extension word formats as exceptions.

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D/A	REGISTER			W/L	0	0	0	DISPLACEMENT INTEGER							

(a) MC68000, MC68008, and MC68010

15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0
D/A	REGISTER			W/L	SCALE		0	DISPLACEMENT INTEGER							

(b) CPU32, MC68020, MC68030, and MC68040

Figure 2-3. M68000 Family Brief Extension Word Formats

2.5 FULL EXTENSION ADDRESSING MODES

The full extension word format provides additional addressing modes for the MC68020, MC68030, and MC68040. There are four elements common to these full extension addressing modes: a base register (BR), an index register (Xn), a base displacement (bd), and an outer displacement (od). Each of these four elements can be suppressed independently of each other. However, at least one element must be active and not suppressed. When an element is suppressed, it has an effective value of zero.

BR can be suppressed through the BS field of the full extension word format. The encoding of bits 0-5 in the single effective address word format (see Figure 2-2) selects BR as either the PC when using program relative addressing modes, or An when using non-program relative addressing modes. The value of the PC is the address of the extension word. For the non-program relative addressing modes, BR is the contents of a selected An.

SIZE and SCALE can be used to modify Xn. The W/L field in the full extension format selects the size of Xn as a word or long word. The SCALE field selects the scaling factor, shifts the value of the Xn left multiplying the value by 1, 2, 4, or 8, respectively, without actually changing the value. Scaling can be used to calculate the address of arrayed structures. Figure 2-4 illustrates the scaling of an Xn.

The bd and od can be either word or long word. The size of od is selected through the encoding of the I/IS field in the full extension word format (refer to Table 2-2). There are two main modes of operation that use these four elements in different ways: no memory indirect action and memory indirect. The od is provided only for using memory indirect addressing modes of which there are three types: with preindex, with postindex, and with index suppressed.

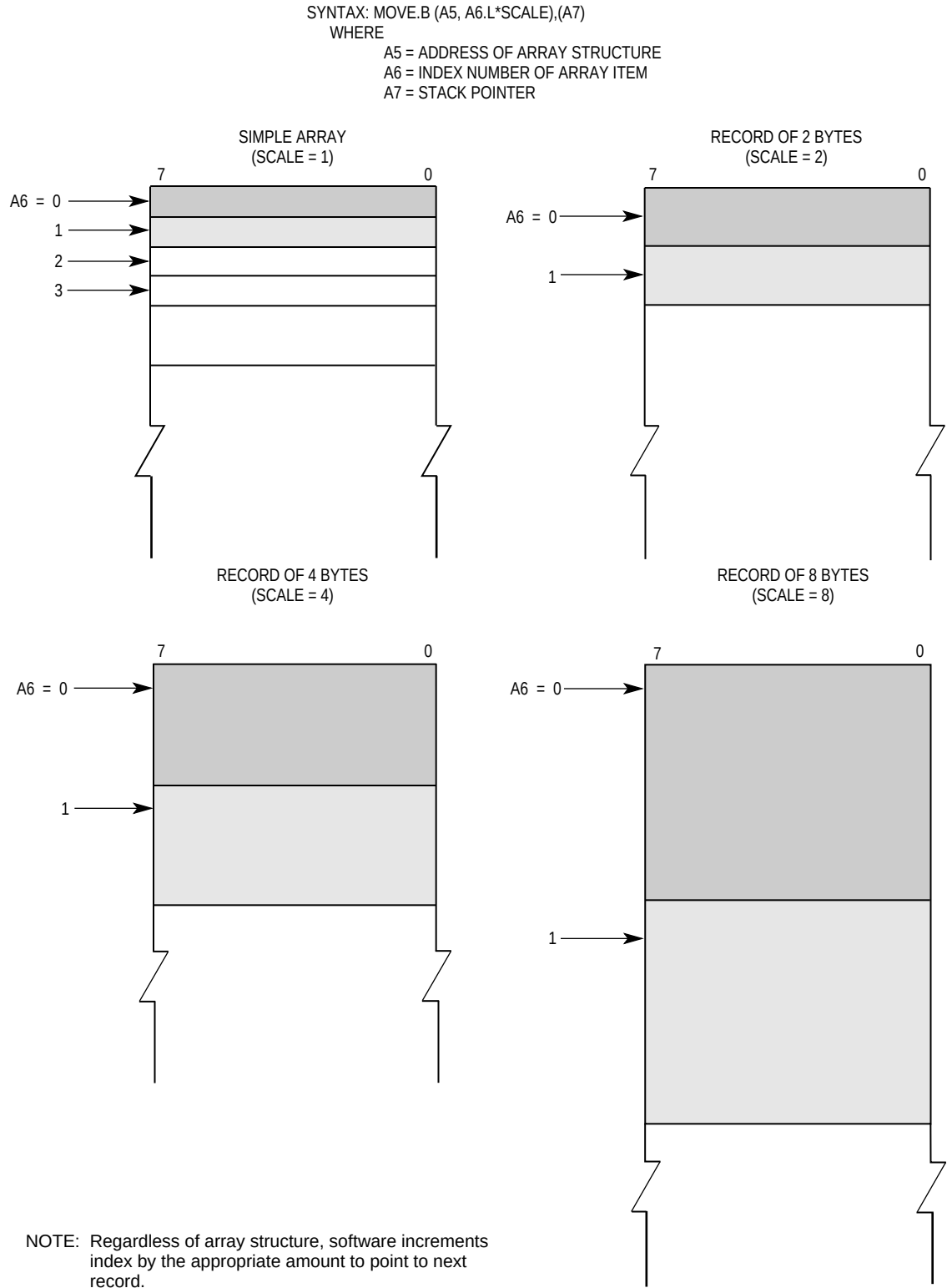


Figure 2-4. Addressing Array Items

2.5.1 No Memory Indirect Action Mode

No memory indirect action mode uses BR, Xn with its modifiers, and bd to calculate the address of the required operand. Data register indirect (Dn) and absolute address with index (bd,Xn.SIZE*SCALE) are examples of the no memory indirect action mode. Figure 2-5 illustrates the no memory indirect action mode.

BR	Xn	bd	Addressing Mode
S	S	S	Not Applicable
S	S	A	Absolute Addressing Mode
S	A	S	Register Indirect
S	A	A	Register Indirect with Constant Index
An	S	S	Address Register Indirect
An	S	A	Address Register Indirect with Constant Index
An	A	S	Address Register Indirect with Variable Index
An	A	A	Address Register Indirect with Constant and Variable Index
PC	S	S	PC Relative
PC	S	A	PC Relative with Constant Index
PC	A	S	PC Relative with Variable Index
PC	A	A	PC Relative with Constant and Variable Index

NOTE: S indicates suppressed and A indicates active.

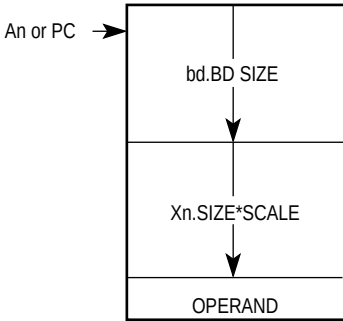


Figure 2-5. No Memory Indirect Action

2.5.2 Memory Indirect Modes

Memory indirect modes fetch two operands from memory. The BR and bd evaluate the address of the first operand, intermediate memory pointer (IMP). The value of IMP and the od evaluates the address of the second operand.

There are three types of memory indirect modes: pre-index, post-index, and index register suppressed. Xn and its modifiers can be allocated to determine either the address of the IMP (pre-index) or to the address of the second operand (post-index).

2.5.2.1 MEMORY INDIRECT WITH PREINDEX. The Xn is allocated to determine the address of the IMP. Figure 2-6 illustrates the memory indirect with pre-indexing mode.

BR	Xn	bd	od	IMP Addressing Mode	Operand Addressing Mode
S	A	S	S	Register Indirect	Memory Pointer Directly to Data Operand
S	A	S	A	Register Indirect	Memory Pointer as Base with Displacement to Data Operand
S	A	A	S	Register Indirect with Constant Index	Memory Pointer Directly to Data Operand
S	A	A	A	Register Indirect with Constant Index	Memory Pointer as Base with Displacement to Data Operand
An	A	S	S	Address Register Indirect with Variable Index	Memory Pointer Directly to Data Operand
An	A	S	A	Address Register Indirect with Variable Index	Memory Pointer as Base with Displacement to Data Operand
An	A	A	S	Address Register Indirect with Constant and Variable Index	Memory Pointer Directly to Data Operand
An	A	A	A	Address Register Indirect with Constant and Variable Index	Memory Pointer as Base with Displacement to Data Operand
PC	A	S	S	PC Relative with Variable Index	Memory Pointer Directly to Data Operand
PC	A	S	A	PC Relative with Variable Index	Memory Pointer as Base with Displacement to Data Operand
PC	A	A	S	PC Relative with Constant and Variable Index	Memory Pointer Directly to Data Operand
PC	A	A	A	PC Relative with Constant and Variable Index	Memory Pointer as Base with Displacement to Data Operand

NOTE: S indicates suppressed and A indicates active.

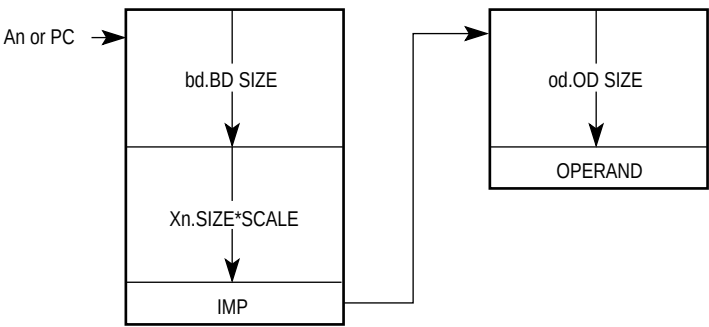


Figure 2-6. Memory Indirect with Preindex

2.5.2.2 MEMORY INDIRECT WITH POSTINDEX. The Xn is allocated to evaluate the address of the second operand. Figure 2-7 illustrates the memory indirect with post-indexing mode.

BR	Xn	bd	od	IMP Addressing Mode	Operand Addressing Mode
S	A	S	S	—	—
S	A	S	A	—	—
S	A	A	S	Absolute Addressing Mode	Memory Pointer with Variable Index to Data Operand
S	A	A	A	Absolute Addressing Mode	Memory Pointer with Constant and Variable Index to Data Operand
An	A	S	S	Address Register Indirect	Memory Pointer with Variable Index to Data Operand
An	A	S	A	Address Register Indirect	Memory Pointer with Constant and Variable Index to Data Operand
An	A	A	S	Address Register Indirect with Constant Index	Memory Pointer with Variable Index to Data Operand
An	A	A	A	Address Register Indirect with Constant Index	Memory Pointer with Constant and Variable Index to Data Operand
PC	A	S	S	PC Relative	Memory Pointer with Variable Index to Data Operand
PC	A	S	A	PC Relative	Memory Pointer with Constant and Variable Index to Data Operand
PC	A	A	S	PC Relative with Constant Index	Memory Pointer with Variable Index to Data Operand
PC	A	A	A	PC Relative with Constant Index	Memory Pointer with Constant and Variable Index to Data Operand

NOTE: S indicates suppressed and A indicates active.

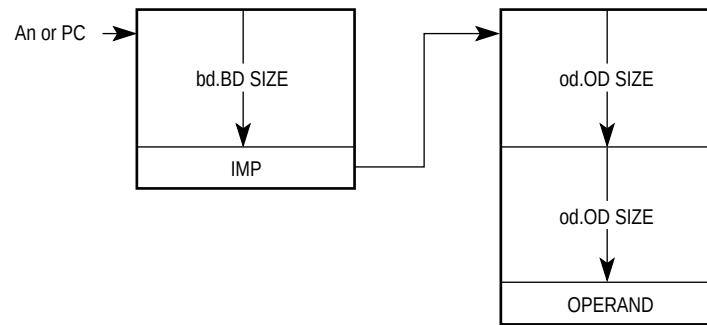


Figure 2-7. Memory Indirect with Postindex

2.5.2.3 MEMORY INDIRECT WITH INDEX SUPPRESSED. The Xn is suppressed. Figure 2-8 illustrates the memory indirect with index suppressed mode.

BR	Xn	bd	od	IMP Addressing Mode	Operand Addressing Mode
S	S	S	S	—	—
S	S	S	A	—	—
S	S	A	S	Absolute Addressing Mode	Memory Pointer Directly to Data Operand
S	S	A	A	Absolute Addressing Mode	Memory Pointer as Base with Displacement to Data Operand
An	S	S	S	Address Register Indirect	Memory Pointer Directly to Data Operand
An	S	S	A	Address Register Indirect	Memory Pointer as Base with Displacement to Data Operand
An	S	A	S	Address Register Indirect with Constant Index	Memory Pointer Directly to Data Operand
An	S	A	A	Address Register Indirect with Constant Index	Memory Pointer as Base with Displacement to Data Operand
PC	S	S	S	PC Relative	Memory Pointer Directly to Data Operand
PC	S	S	A	PC Relative	Memory Pointer as Base with Displacement to Data Operand
PC	S	A	S	PC Relative with Constant Index	Memory Pointer Directly to Data Operand
PC	S	A	A	PC Relative with Constant Index	Memory Pointer as Base with Displacement to Data Operand

NOTE: S indicates suppressed and A indicates active.

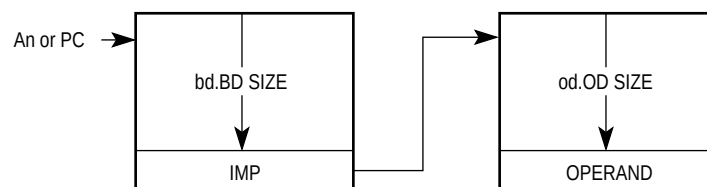


Figure 2-8. Memory Indirect with Index Suppress

2.6 OTHER DATA STRUCTURES

Stacks and queues are common data structures. The M68000 family implements a system stack and instructions that support user stacks and queues.

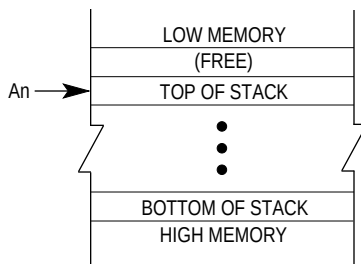
2.6.1 System Stack

Address register seven (A7) is the system stack pointer. Either the user stack pointer (USP), the interrupt stack pointer (ISP), or the master stack pointer (MSP) is active at any one time. Refer to **Section 1 Introduction** for details on these stack pointers. To keep data on the system stack aligned for maximum efficiency, the active stack pointer is automatically decremented or incremented by two for all byte-size operands moved to or from the stack. In long-word-organized memory, aligning the stack pointer on a long-word address significantly increases the efficiency of stacking exception frames, subroutine calls and returns, and other stacking operations.

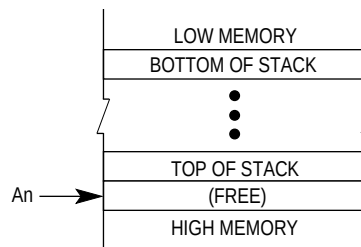
The user can implement stacks with the address register indirect with postincrement and predecrement addressing modes. With an address register the user can implement a stack that fills either from high memory to low memory or from low memory to high memory. Important consideration are:

- Use the predecrement mode to decrement the register before using its contents as the pointer to the stack.
- Use the postincrement mode to increment the register after using its contents as the pointer to the stack.
- Maintain the stack pointer correctly when byte, word, and long-word items mix in these stacks.

To implement stack growth from high memory to low memory, use $-(A_n)$ to push data on the stack and $(A_n) +$ to pull data from the stack. For this type of stack, after either a push or a pull operation, the address register points to the top item on the stack.



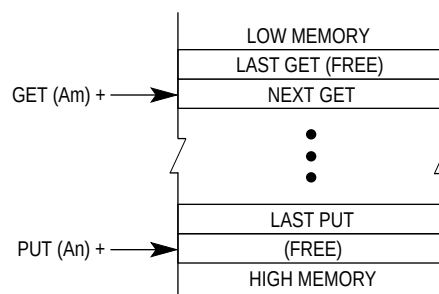
To implement stack growth from low memory to high memory, use $(An) +$ to push data on the stack and $-(An)$ to pull data from the stack. After either a push or pull operation, the address register points to the next available space on the stack. .



2.6.2 Queues

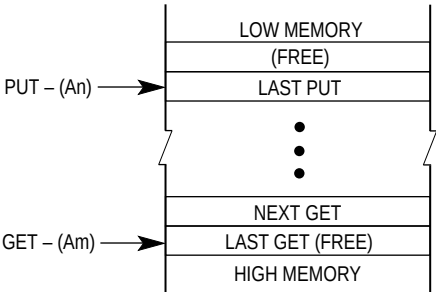
The user can implement queues, groups of information waiting to be processed, with the address register indirect with postincrement or predecrement addressing modes. Using a pair of address registers, the user implements a queue that fills either from high memory to low memory or from low memory to high memory. Two registers are used because the queues get pushed from one end and pulled from the other. One address register contains the put pointer; the other register the get pointer. To implement growth of the queue from low memory to high memory, use the put address register to put data into the queue and the get address register to get data from the queue.

After a put operation, the put address register points to the next available space in the queue; the unchanged get address register points to the next item to be removed from the queue. After a get operation, the get address register points to the next item to be removed from the queue; the unchanged put address register points to the next available space in the queue. .



To implement the queue as a circular buffer, the relevant address register should be checked and adjusted. If necessary, do this before performing the put or get operation. Subtracting the buffer length (in bytes) from the register adjusts the address register. To implement growth of the queue from high memory to low memory, use the put address register indirect to put data into the queue and get address register indirect to get data from the queue.

After a put operation, the put address register points to the last item placed in the queue; the unchanged get address register points to the last item removed from the queue. After a get operation, the get address register points to the last item placed in the queue.



To implement the queue as a circular buffer, the get or put operation should be performed first. Then the relevant address register should be checked and adjusted, if necessary. Adding the buffer length (in bytes) to the address register contents adjusts the address register.